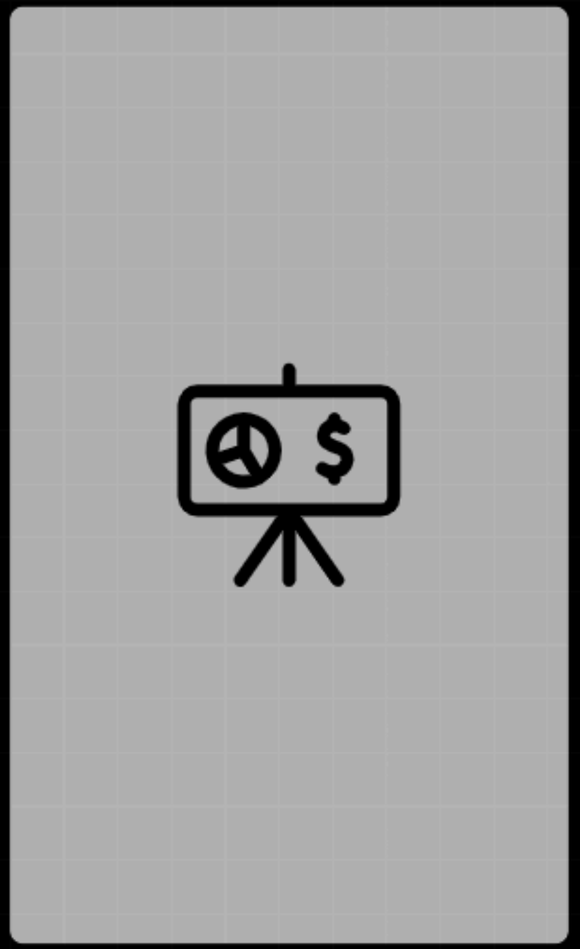
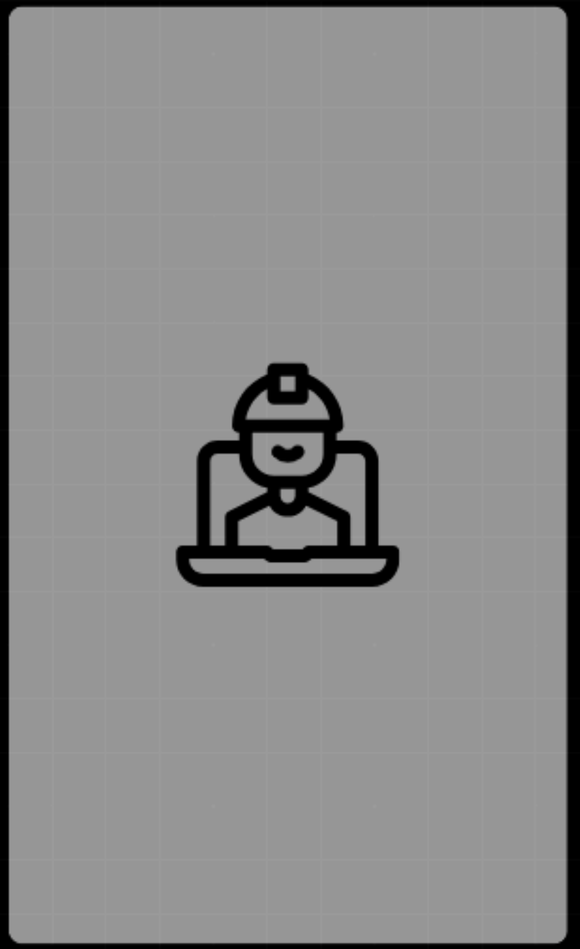
Day 19 - OpenAI Agents SDK

Transitioning from basic agent concepts to framework implementation with the newest and most lightweight agent framework from OpenAI
Prerequisites: Understanding async/await patterns in Python for building practical multi-agent systems.

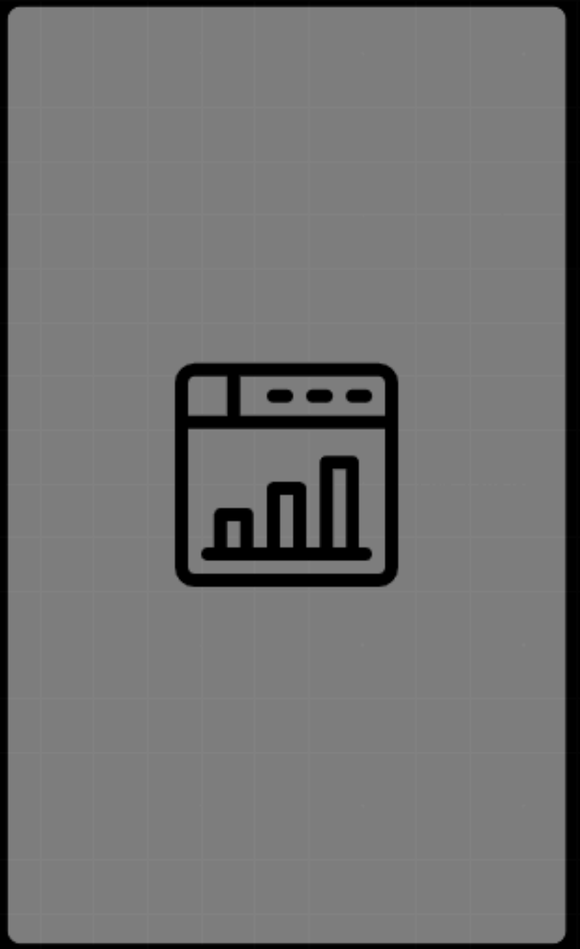
The Foundation - Asynchronous Python



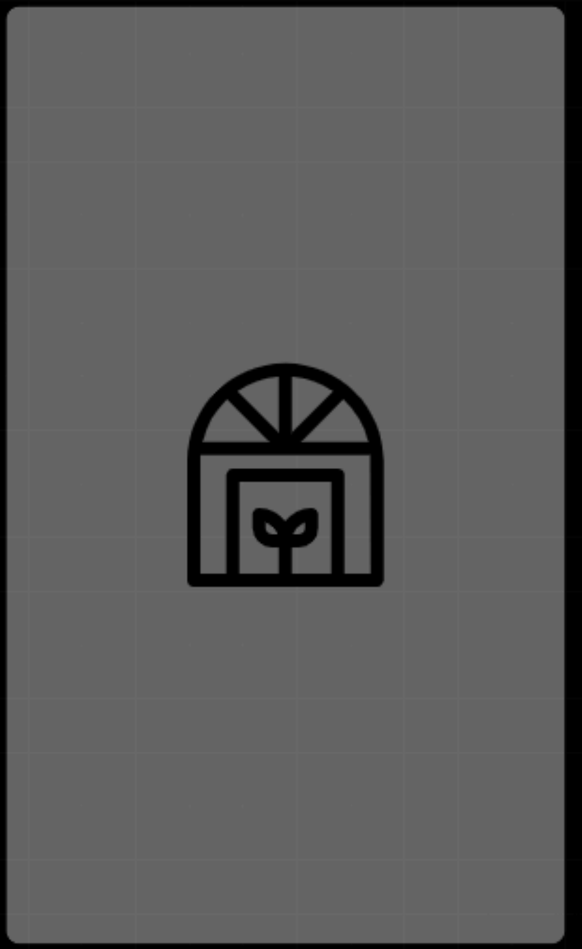
All major **agent frameworks** use async IO



Critical for understanding **agent orchestration**



Not optional - fundamental to modern agent development

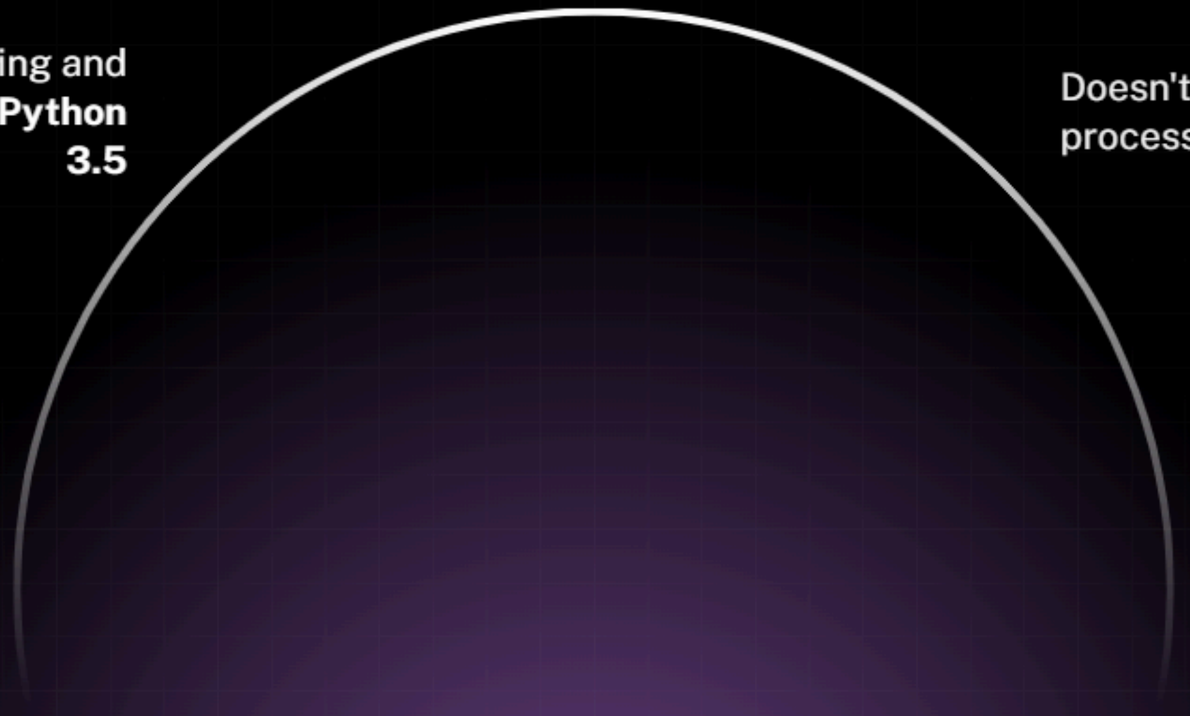


Takes 30 minutes to master, saves hours of confusion

Async IO - Lightweight Concurrency

Alternative to multithreading and multiprocessing introduced in **Python 3.5**

Doesn't use OS-level threads or multiple processes



Allows thousands of concurrent operations with minimal resources

Perfect for **I/O-bound operations** like API calls and network requests

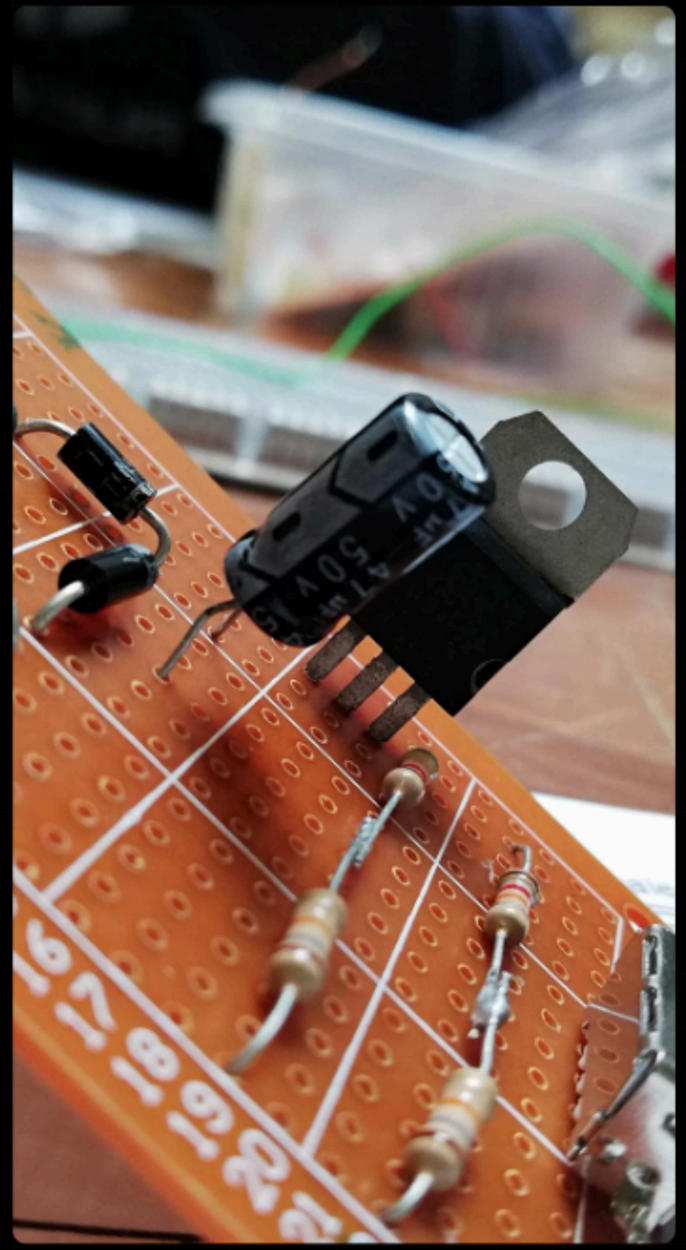
Async IO in Two Keywords

async - Declares a coroutine

Place before function definition. Example syntax: async def process_data()

await - Schedules execution

Place before calling async function. Example syntax: result = await process_data()



Understanding Coroutines

1

Regular Function

Executes immediately when called and runs until completion or return.

2

Coroutine

TRIAL

Returns a coroutine object; can be paused and resumed; managed by event loop.

How Async Python Actually Works

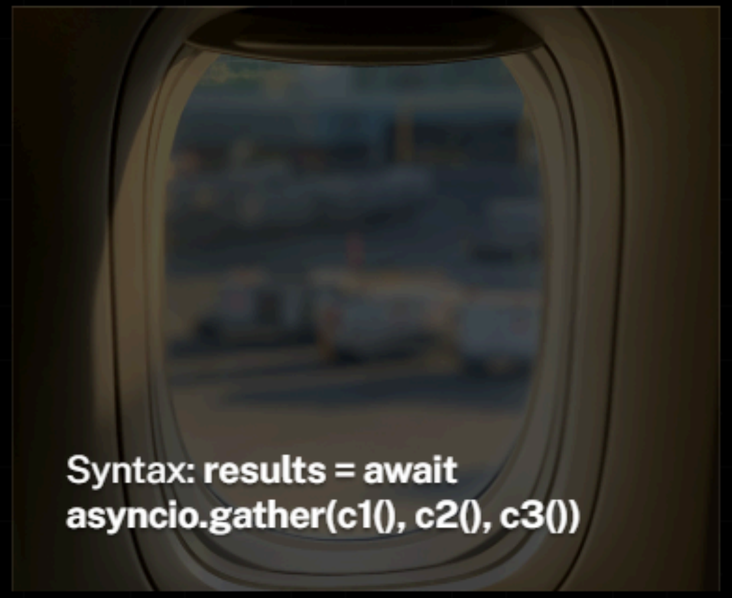
The Event Loop

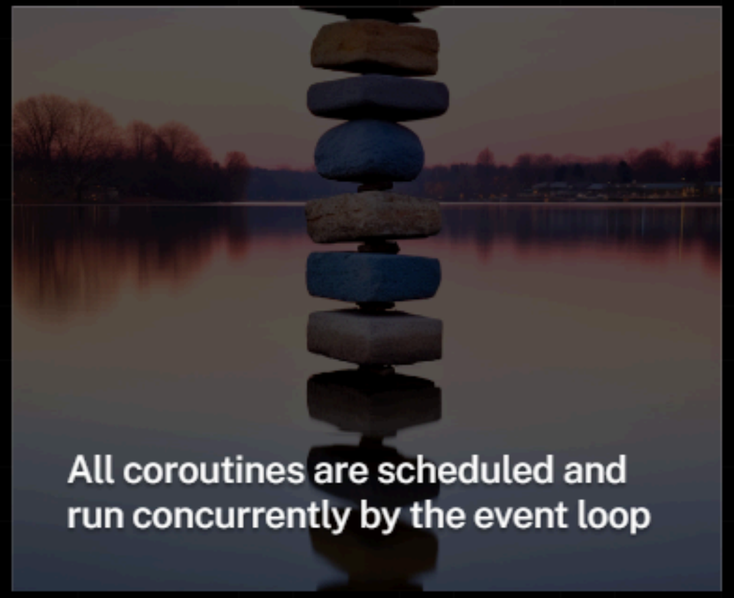
Maintains a queue of coroutines; executes one at a time and switches during I/O.

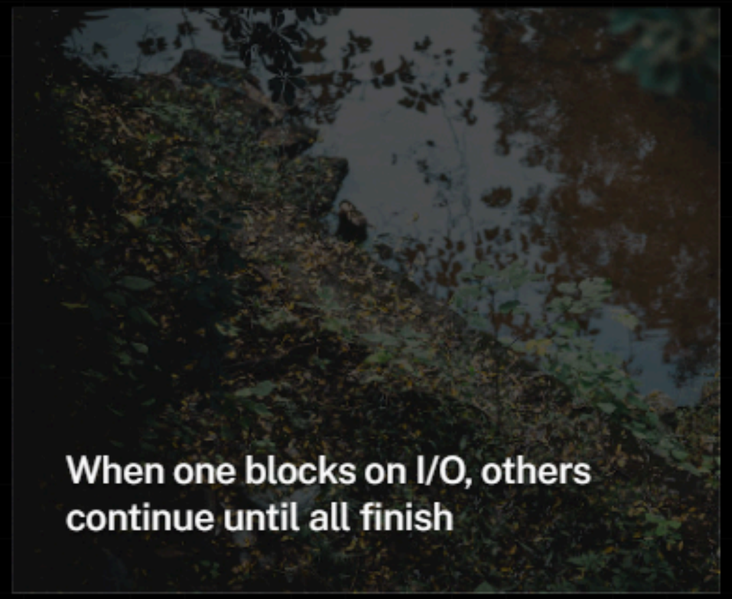
Lightweight Nature

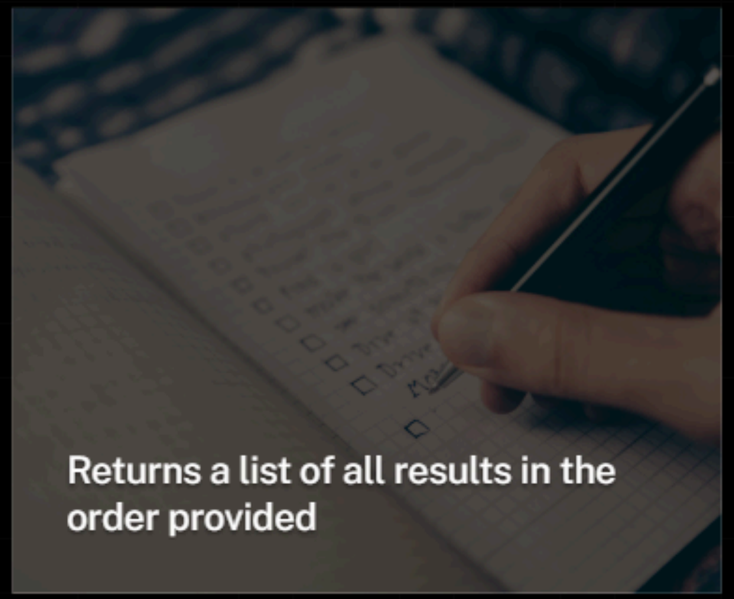
All managed in Python without OS threads; can handle **10,000+** concurrent operations.

Concurrent Execution with gather









Async IO and Agent Frameworks

- Agents often call LLMs repeatedly, involving heavy network waiting
- Different agents can work simultaneously in **Multi-Agent Systems**
- Coordinate without blocking each other for efficient resource use
- Scale to handle hundreds of interactions with minimal memory overhead

OpenAI Agents SDK Overview

Lightweight And Flexible - Not Opinionated, Gives You Full Control

Handles Boilerplate Tasks Like **JSON** And Tool Definitions

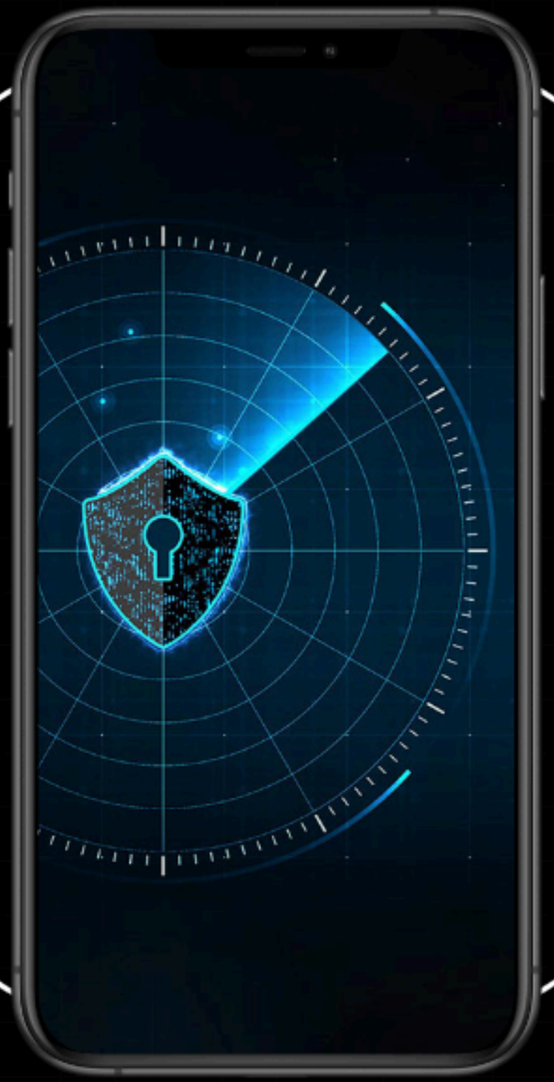
Features Built-In Tracing, Monitoring, And Agent Coordination

Works Seamlessly With Multiple **LLM Providers**

Understanding Agents

Collaborates as a part of a larger multi-agent system

Can have tools and capabilities to perform specialized tasks



A package around **LLM calls** with a specific role and purpose

Defined by a name and instructions (**system prompt**)

Agent Handoffs

Interaction involving the **delegation of control** between agents

Control doesn't return to the original agent once passed

Next agent continues the workflow until the process finishes

Differs from tools as it transfers full **ownership** of the task

Agent Guardrails

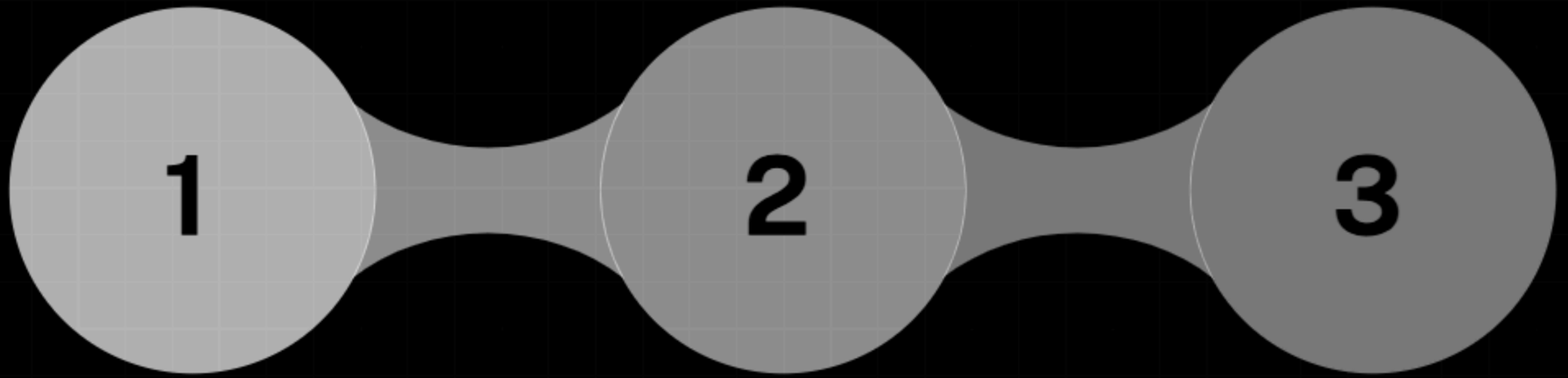
Checks and controls to ensure agents stay on task

Input guardrails validate all incoming data

Output guardrails check generated responses before delivery

Behavioral constraints limit potential agent actions

Basic Agent Execution



Create an Agent

Instantiate the Agent class with a name, instructions, and specified model.

Use Trace

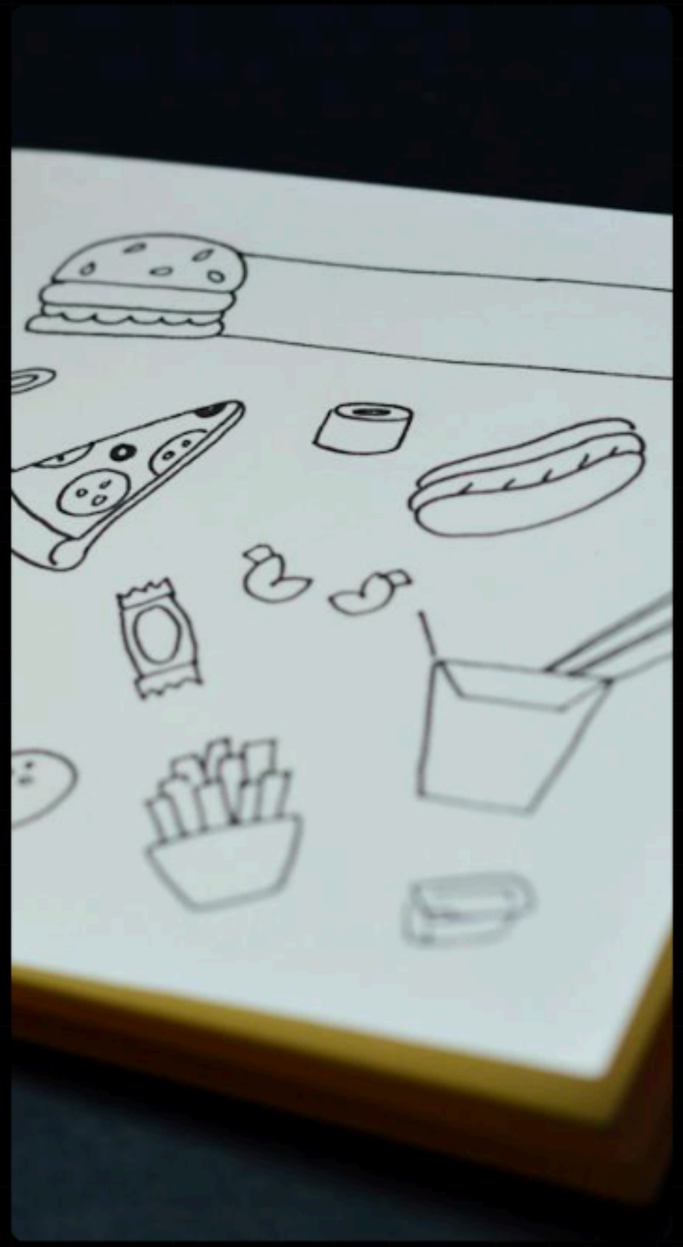
Wrap execution in a trace context to log all interactions for monitoring.

Call runner.run

Execute the agent using the await keyword as it is an asynchronous function.

Key Imports from OpenAI

- **Agent** - The Main Class For Creating Agent Instances
- **Runner** - The Core Utility That Executes Agent Tasks
- **Trace** - Used To Monitor And Log All Interactions
- Install Via The Generic `Openai` Package Module

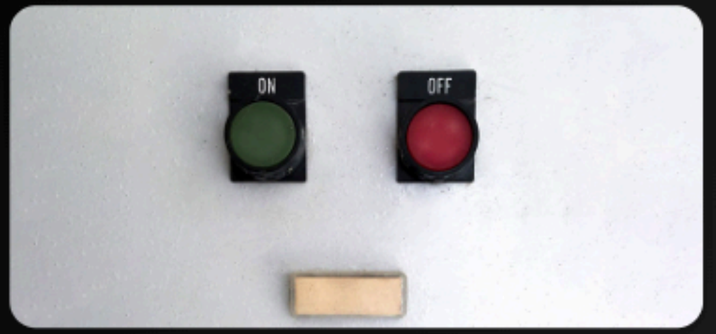


Agent Creation Pattern

Name: Unique identifier (e.g., 'jokester', 'sales_agent_1')

Instructions: System prompt defining tone and behavior

Model: LLM selection (e.g., **gpt-4o-mini**)



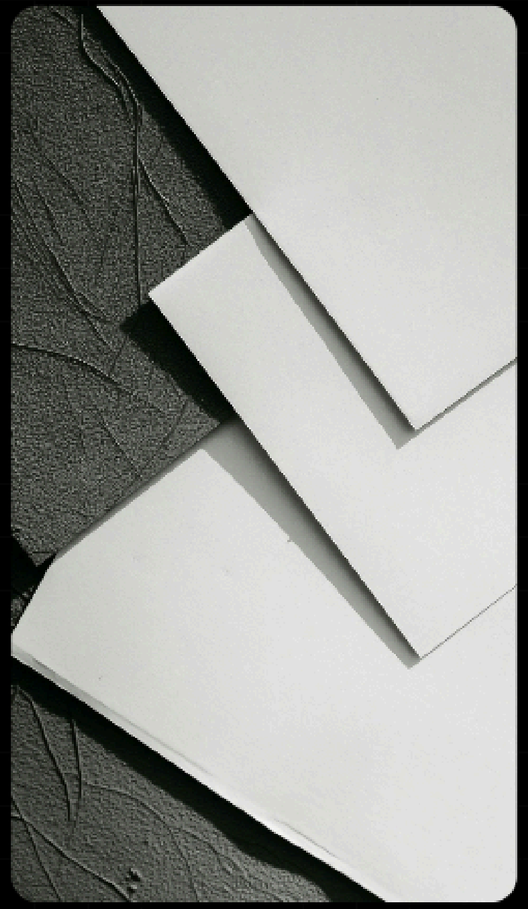
Optional configuration for tools, handoffs, and guardrails

Executing Agents

Call `runner.run()` with the agent instance and user input

Must use **await** keyword as it is a coroutine function

Access results via the `final_output` attribute on response



Returns a response object containing the agent's full reply

Monitoring with Traces

- Wrap execution in a `with trace("name"):` block for grouping
- Records entire workflow, timing, and **token usage** data
- Crucial for debugging complex multi-agent system interactions
- View reports online at **platform.openai.com/traces**

Real-Time Agent Output

Use `runner.run_streamed()` for incremental text delivery

Iterate through chunks using an `async for` loop pattern

Significantly improves user experience and perceived responsiveness

Allows start of processing on partial results immediately

Running Multiple Agents Concurrently

- Use `Asyncio.Gather()` With Multiple Runner Calls For Parallel Work
- All Agents Execute Simultaneously Without Blocking Each Other
- Collect Final Results As A List For Comparison Or Aggregation
- Achieve Up To **3x Faster** Execution Than Sequential Patterns

Coding with LLM Assistance

1. Craft reusable prompts and mention the current date
2. Ask multiple **LLMs** the same question to verify accuracy
3. Generate code in small **10-line chunks** for better control
4. Have a second LLM review the first agent's proposed answer
5. Request **3 different approaches** to find the optimal solution

Avoiding LLM-Assisted Pitfalls

1. Don't generate 200 lines at once; break into small steps
2. Always ask for a step-by-step plan before code generation
3. Ask LLMs to explain unclear parts so you **understand the code**
4. Cross-reference answers using both **ChatGPT** and **Claude**

Automated SDR Architecture

Workflow Layer

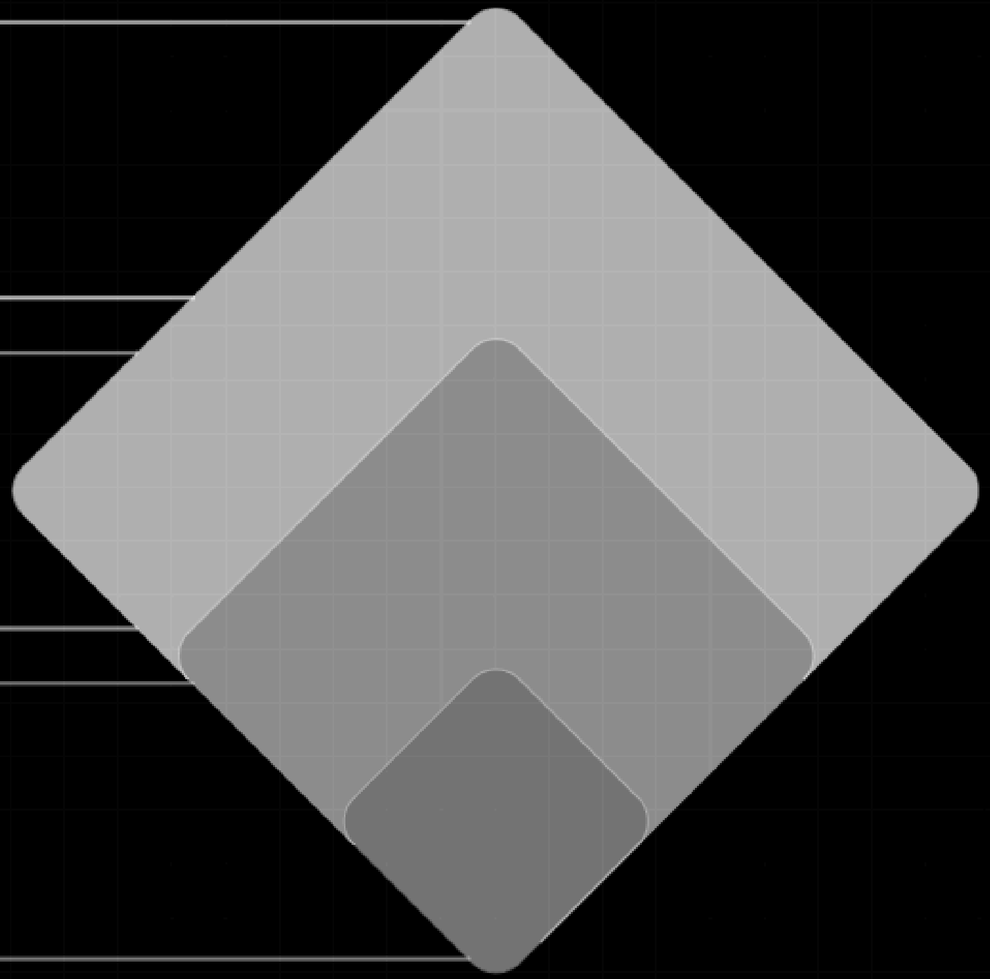
Simple agent calls orchestrated manually in Python code for control flow.

Tools Layer

Agents empowered with functions to perform real-world tasks and lookups.

Collaboration Layer

Agents calling other agents using patterns like tools vs. handoffs.



Manual Agent Orchestration

1 Create multiple agents with distinct instructions for different styles

2 Use `asyncio.gather()` to write emails in parallel concurrently

3 Implement a fourth agent to pick the best output from the group

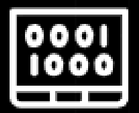
4 Pattern: This is the **evaluator-optimizer** design pattern

Simplified Tool Definition

No more manual **JSON schemas** or complex parameter definitions

Decorate functions with `@function_tool` for automatic schema

SDK reads function signatures and **docstrings** to build tools



Write normal Python functions and get agent tools automatically

Creating Tools from Functions

Just write clean Python with high-quality documentation

Docstring becomes the official tool description for the LLM

The SDK creates a **FunctionTool** object and execution wrapper

Type hints are used to generate the parameter JSON schema

Wrapping Agents for Reuse

1

Concept: Any existing agent instance can become a tool

2

Call **.as_tool()** to create a FunctionTool wrapper

3

When the tool is called, it triggers the underlying agent runner

4

Allows manager agents to choose and call other sub-agents

Two Ways for Agents to Collaborate

Tools: Request-Response where control returns to the caller

Handoffs: Delegation where control does NOT return

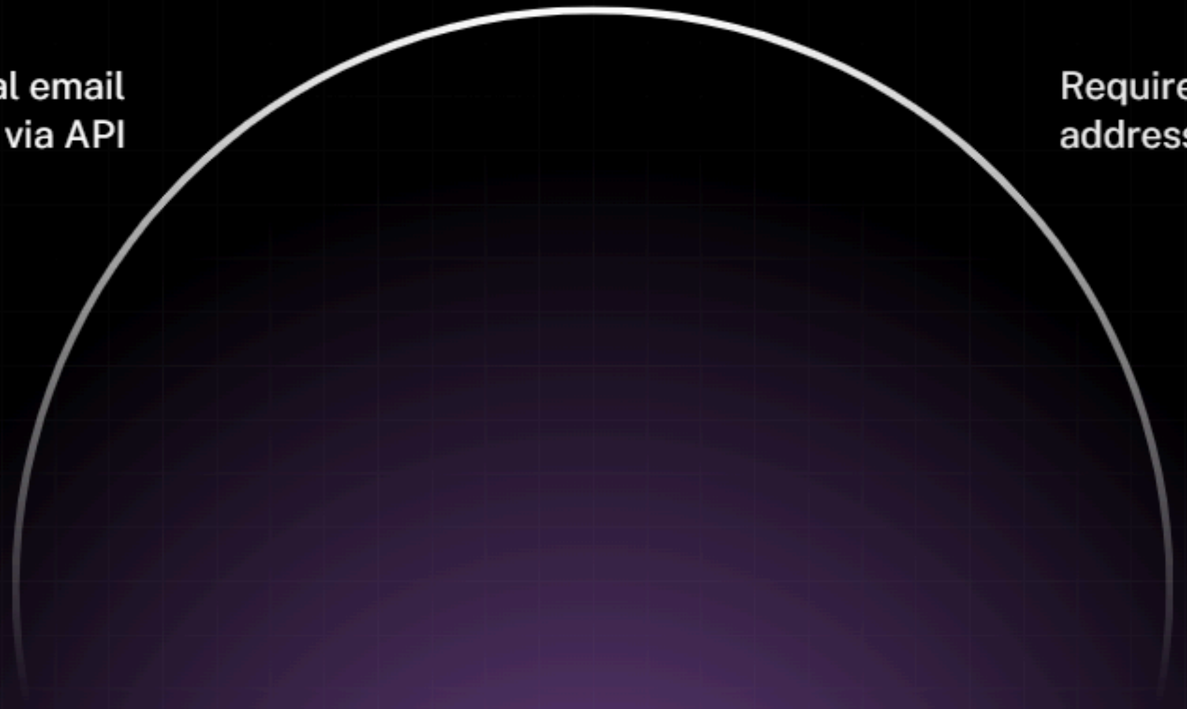
Use tools for giving agents specific capabilities

Use handoffs for task delegation to specialized agents

External Tool Integration

Use **SendGrid** for transactional email automation via API

Requires an API key and verified sender address configuration



Enables agents to perform real-world actions like sending mail

Demonstrates pattern for any external **API integration**

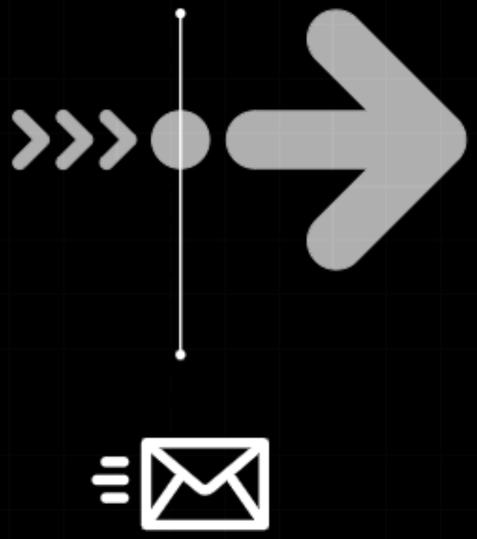
Multi-Agent System with Tools

- Three Sales Agents Wrapped As Tools For Style Variety
- Sales Manager Coordinates And Picks The Best Email Content
- Instruction: Never Generate Emails Yourself, Use Tools Only
- Agent Makes **Autonomous Decisions** About Which Tools To Use

Advanced Multi-Agent Architecture

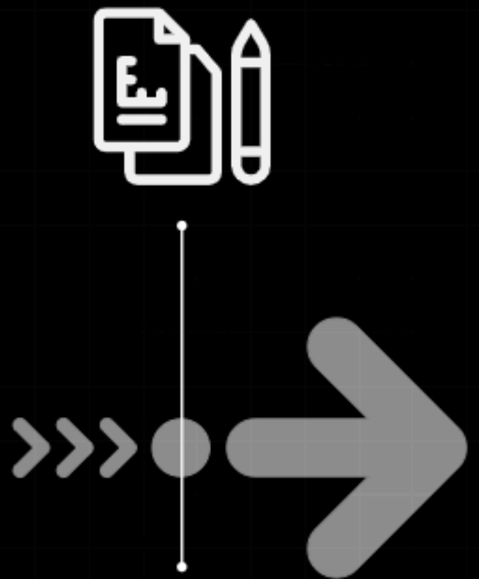
Generate Emails

Sales Manager uses three style-specific agent tools to write drafts.



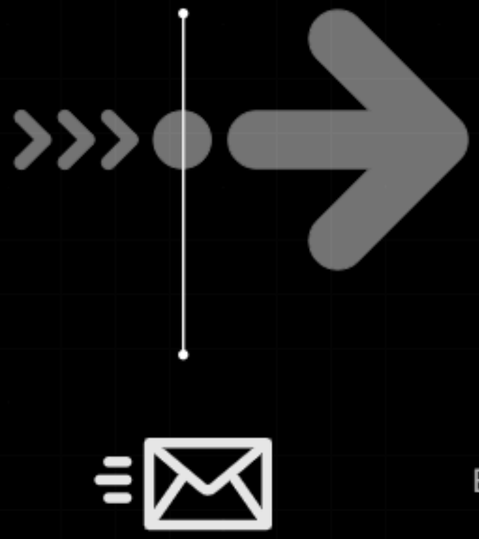
Pick Best Draft

The Manager agent evaluates the outputs and selects the single best email.



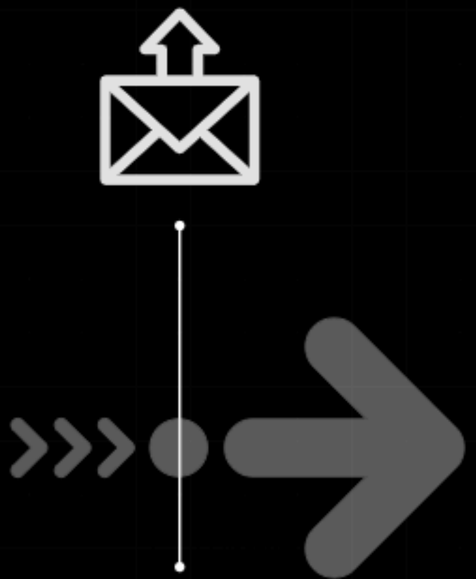
Hand off to Email Manager

Control is delegated to the specialized Email Manager for final processing.

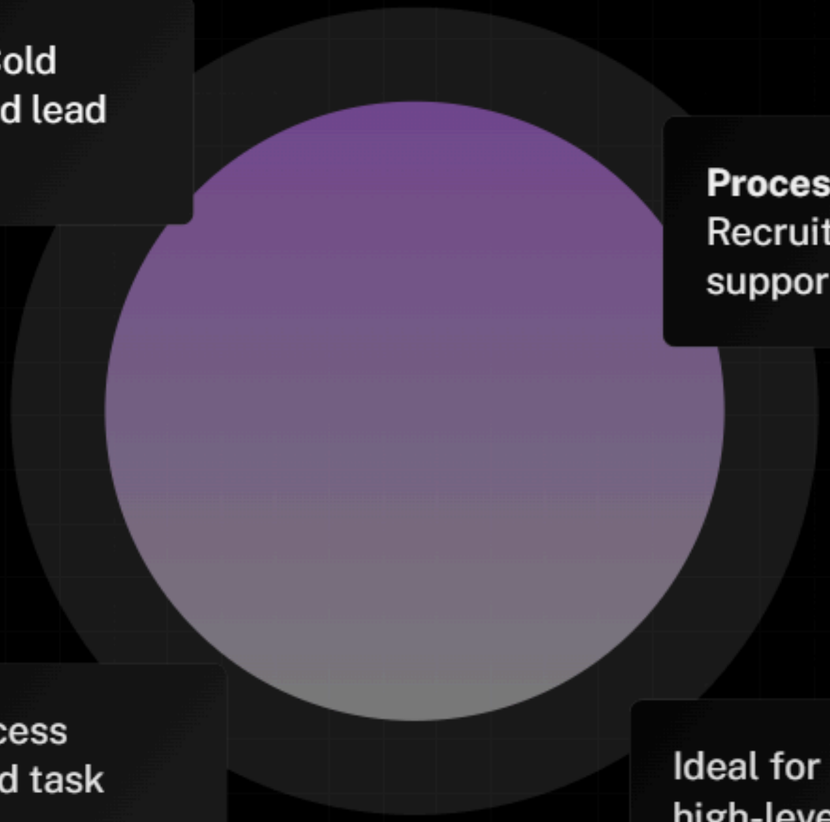


Format and Send

Email Manager uses formatting tools and SendGrid to deliver the final HTML mail.



Real-World Use Cases



Sales Automation: Cold email, follow-ups, and lead qualification

Process Automation: Recruitment and customer support routing

Applies to any process needing specialized task delegation

Ideal for systems requiring high-level **autonomous operation**